CHE 565 -- Homework 5

Soki Sem

April 27, 2026

Introduction

The following homework was done with Simulink and the Control Systems Library in Python. The block diagram of the system is shown in Figure 1. The block diagram was created in simulink and then I built the same diagram as a python function using the control library.

```
import control as ct

def build_closed_loop(Kc1, Kc2, tauI1, tauI2):

    s = ct.tf('s')
    I1 = 1.0 / tauI1
    I2 = 0

    Gc1 = Kc1 * (1 + I1 / s)
    Gc2 = Kc2 * (1 + I2 / s)
    Gp1 = Kp1 / (taup1 * s + 1)
    Gp2 = Kp2 / (taup2 * s + 1)
    Gd = 1 * (1 + 0 * s)    # direct disturbance addition
    numD,denD = ct.delay.pade(theta, pade_order)
    # Named blocks
    Gc1_blk = ct.tf(Gc1, name='Gc1', inputs='E1', outputs='Yc1')
    Gc2_blk = ct.tf(Gc2, name='Gc2', inputs='E2', outputs='Yc2')
    Gp1_blk = ct.tf(Gp1, name='Gp1', inputs='Yc2', outputs='Yp1')
    Gp2_blk = ct.tf(Gp2, name='Gp2', inputs='P', outputs='Yp2')
    Gd_blk = ct.tf(Gd, name='Gd', inputs='D', outputs='Yd')      # direct disturbance additi
    GD_blk = ct.tf(numD, denD, name='GD', inputs='Yp2', outputs='Y')
    sum1 = ct.summing_junction(inputs=['Ysp', '-Yp2'], output='E1', name='Sum1')
    sum2 = ct.summing_junction(inputs=['Yc1'], output='E2', name='Sum2')
    sum3 = ct.summing_junction(inputs=['Yp1', 'Yd'], output='P', name='Sum3')
    sys = ct.interconnect(
        [Gc1_blk, Gc2_blk, Gp1_blk, Gp2_blk, Gd_blk, sum1, sum2, sum3],
        input=['Ysp', 'D'],
        output=['Yp2'],
        input_prefix = ["Ysp", "D"],
        output_prefix = ["Yp2"],
    )
    return sys

def calculate_IAE(t, y, ysp):
    error = ysp - y
    iae = np.trapezoid(np.abs(error), t)
```

```
    return iae

# ------------------------------------------------
# Simulation helper
# ------------------------------------------------
def simulate_case(sys, t, ysp_input, d_input):
    U = np.vstack([ysp_input, d_input])
    resp = ct.forced_response(sys, T=t, U=U, squeeze=True)
    return resp.time, resp.outputs
%
```

Problem 1

Closed-loop Response to Step Disturbance

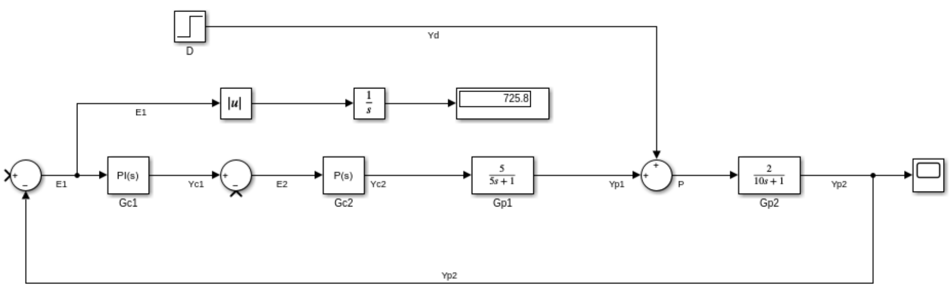
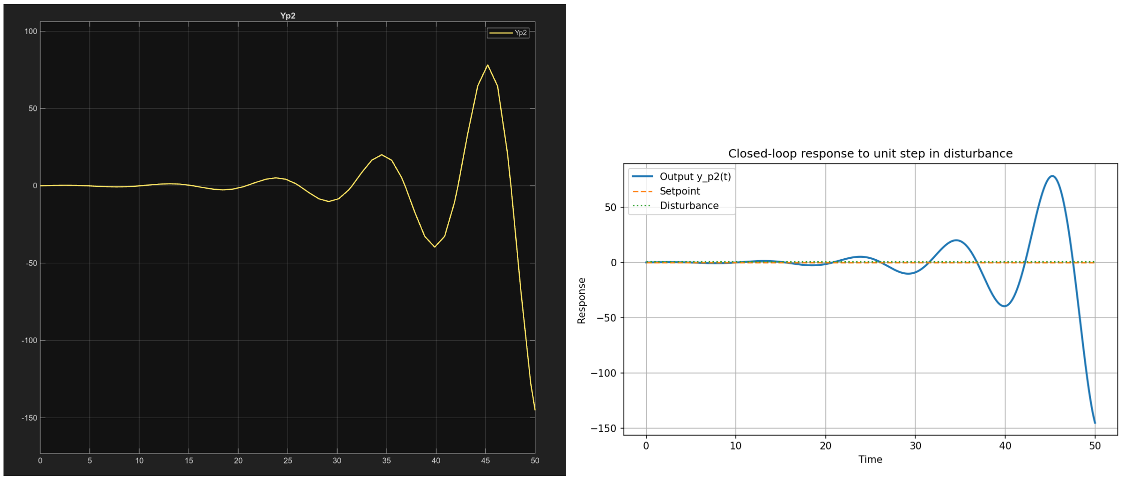


Figure 1: Block diagram of the closed-loop system without cascade control and with a unit step disturbance from Simulink.

The Simulink block diagram is shown in Figure 1.

Case 1: Step disturbance with no setpoint change gave and step in disturbance with no setpoint change gaveIAE = 725.3696 in Python and IAE = 725.8109 in Simulink.



(a) Simulink

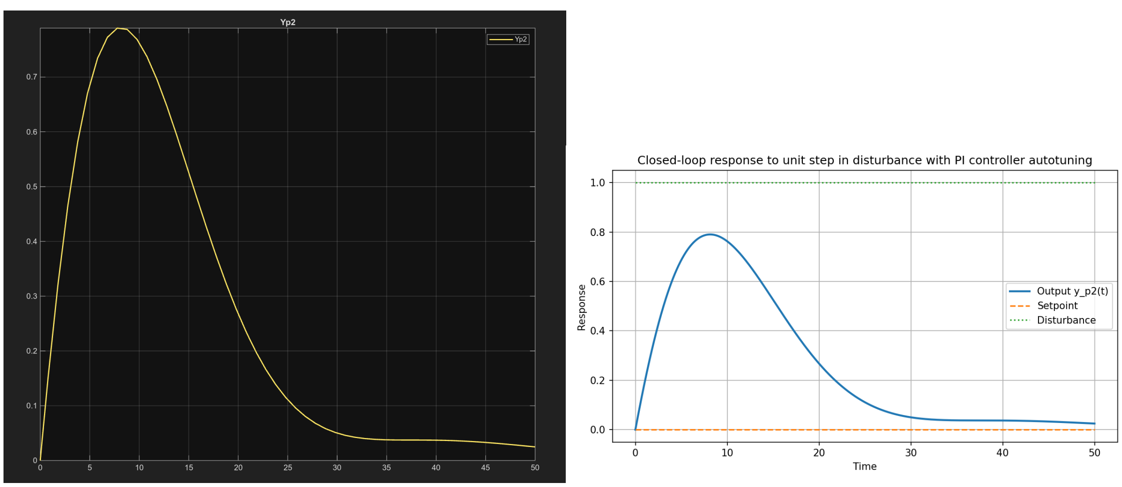
(b) Python

Figure 2: Closed-loop response to no step change in setpoint and a unit step change in disturbance with no cascade control and no autotuning.

The closed-loop disturbance response and no cascade loop is shown in Figure 2.

The PI controller was then tuned using the autotuning feature in Simulink, which gave P = 0.1837 and I = 0.0816.

Case 2: Step disturbance with no setpoint change and PI autotuning gave IAE = 13.0462 in Python and IAE = 13.0463 in Simulink.



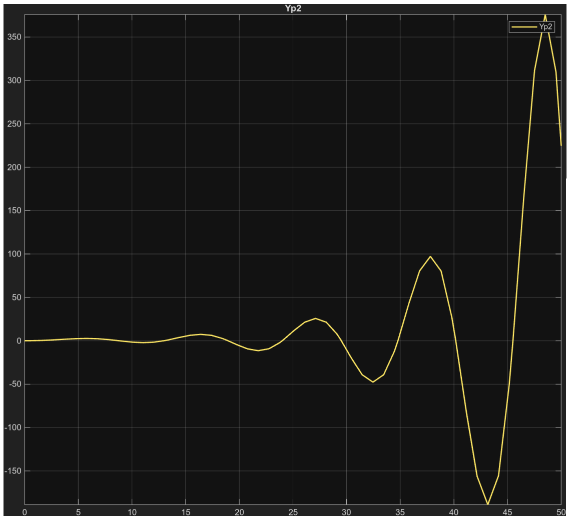
(a) Simulink

(b) Python

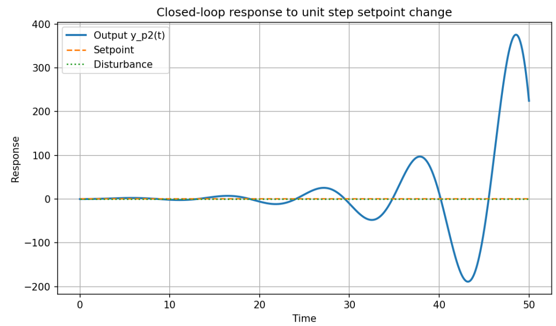
Figure 3: Closed-loop response to no step change in setpoint and a unit step change in disturbance with autotuning.

The response to a unit step change in disturbance with no cascade control and PI autotuning is shown in Figure 3.

Closed-loop Response to Step Setpoint Change



(a) Simulink

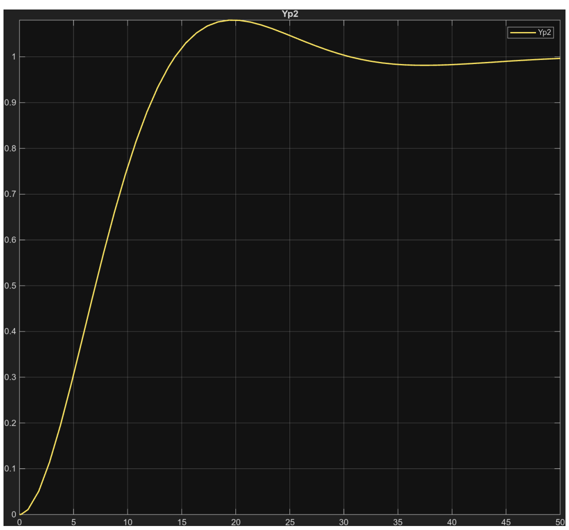


(b) Python

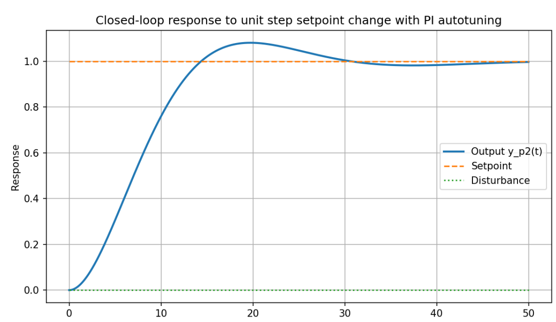
Figure 4: Closed-loop response to step change in setpoint and no step disturbance.

The response to the step change in setpoint with no step disturbance is shown in Figure 4.

Case 3: Step setpoint change with no disturbance change gave IAE = 2449.4065 in Python and IAE = 2450.8205 in Simulink.



(a) Simulink



(b) Python

Figure 5: Closed-loop response to step change in setpoint and no step disturbance with PI autotuning.

The response to a unit step change in setpoint with no disturbance change using PI autotuning is shown in Figure 5.

Case 4: Step setpoint change with no disturbance change and PI autotuning gave IAE = 8.1939 in Python and IAE = 8.1939 in Simulink.